

STENCIL

AI-Aware Backend Code Generation Platform

Product Specification · v5.0 · April 2026 · Confidential

1. Executive Summary

Stencil is a deterministic, AI-aware backend code generation platform built around a two-layer architecture: core infrastructure declares what the system talks to; API resources declare what is exposed to the outside world. From a single YAML spec, Stencil generates 100% of structural code for a production-ready backend service.

Core principle

Core infra (tables, externals, messaging, storage, cache, transactions) defines everything the service touches. API resources group related endpoints. Each API declares which infra it touches and in what order. The tool generates a shared context object and hook interface per API. Developers implement hooks to set control flags and inject business logic. Nothing else needs to be written.

Token reduction

AI writes a compact YAML spec. Stencil generates all structural code deterministically. AI only implements hook functions containing business logic. Token usage drops by an estimated 75-85% for typical backend development.

Key principles

- Two layers — core infra declares existence; API resources declare exposure
- Shared context — one typed object flows through every step of an API execution
- Control flags — boolean fields in shared context control which steps run
- Hook interface — generated per API, one Before/After hook per infra touch
- Developer owns flags — hooks set and unset flags to express business logic
- Deterministic — identical spec always produces identical code
- Immutable generated code — generated/ is read-only, never edited
- Multi-language — Go v1, Java Spring Boot v2

2. The Two-Layer Model

2.1 Layer 1 — Core Infrastructure

Core infra blocks define everything the service talks to. They are independent of any API — they declare what exists and how to interact with it. The tool generates fully typed interfaces for each.

DSL block	What it declares	Tool generates
tables	DB tables — fields, queries, indexes, state machines, soft delete	Migration SQL, typed model struct, repository functions, sqlc queries
types	Custom value objects — Money, Address, GeoPoint	Typed struct + Validate() in generated/types/
transactions	Multi-step atomic DB operations	BEGIN/COMMIT/ROLLBACK orchestrator function
externals	Outbound HTTP — third-party APIs and inter-service calls	Typed client + retry + error mapping + mock
messaging	Kafka/RabbitMQ/SQS producers and consumers	Typed publish functions, consumer listener + DLQ
storage	S3/GCS object storage + upload definitions	Upload endpoint, validation, S3 put, field update
cache	Redis typed cache interfaces per resource	Get/Set/Delete/Invalidate methods with key templates
auth	JWT config, roles, permissions, login flows	Auth middleware, login/logout/refresh endpoints
observability	Metrics, tracing, structured logging	Auto-instrumented middleware on every route

2.2 Layer 2 — API Resources

API resources declare what is exposed externally. Resources are groups of related endpoints. Each endpoint declares which core infra it touches, in what order, and with what default control flags. The tool generates the shared context, service executor, and hook interface for each API.

Resources are not tables

In the old model, a resource implied a DB table. In Stencil, a resource is an API group — a set of related endpoints around a domain concept. An endpoint may touch zero, one, or many infra

systems. ChargeCard touches Stripe and Kafka but no DB table. GetUser touches the cache first, then the DB. CreateUser touches the DB, a CRM external, and Kafka. The API declaration drives everything.

2.3 The Shared Context

Every API execution has one shared context object that flows from start to finish. It is created at request entry, flows through every generated step and every developer hook, and holds the final response. It contains three categories of fields:

Category	Contents
Request	The parsed, validated incoming request. Hooks can mutate fields (e.g. replace plain password with hash).
Control flags	One boolean per infra touch. Default set at context creation from spec declaration. Hooks set/unset to control which steps execute.
Infra results	One slot per infra touch for the result. Set by generated code after each step runs. Available to all subsequent hooks.
Infra errors	One error slot per infra touch. Hook can inspect and decide to treat as fatal or non-fatal.
Response	The outgoing response. Developer sets this in BeforeResponse hook. If nil, tool builds it from TableResult.

The shared context is the state machine. The generated service is the executor. Developer hooks are the transitions.

Generated code never makes decisions — it only reads control flags. All decisions live in developer hooks. All state lives in the shared context. This is the complete separation between generated structure and developer logic.

3. DSL Full Schema Reference

This section documents every keyword in stencil.yaml. The spec is validated before any code is generated. All errors are reported in a single pass with line numbers.

3.1 Top-Level Structure

```

version:      1                # (req) DSL schema version
project:      orders-service   # (req) str - module/artifact name
lang:         go               # (req) enum[go, java]
framework:    gin              # (req) enum[gin, echo, fiber] | enum[spring]
db:           postgres         # (req) enum[postgres, mysql, mongo]

# — Core Infrastructure —————
config:       [...]           # (req) environment variables
tables:       [...]           # (opt) DB tables
types:        [...]           # (opt) custom value objects
transactions: [...]           # (opt) atomic multi-step DB ops
externals:     [...]           # (opt) outbound HTTP clients
messaging:    {...}           # (opt) Kafka/RabbitMQ/SQS
storage:      {...}           # (opt) S3/GCS object storage
cache:        {...}           # (opt) Redis typed interfaces
auth:         {...}           # (opt) JWT, roles, permissions
observability:{...}           # (opt) metrics, tracing, logging
middleware:   [...]           # (opt) global middleware stack

# — API Layer —————
resources:    [...]           # (req) API groups

# — Escape Hatches —————
extensions:   [...]           # (opt) full subsystems outside DSL
overrides:    {...}           # (opt) global behaviour overrides

```

3.2 Tables Block

Declares all DB tables. Tables are pure infrastructure — they have no knowledge of APIs. Multiple APIs can touch the same table.

```

tables:
  - name: users                # (req) str - table name (snake_case)
    fields:                    # (req) list<Field>
      - name: email
        type: str              #
    enum[str,int,decimal,bool,date,timestamp,json,uuid,enum]
    required: true              # NOT NULL + required validator
    unique:   true              # UNIQUE INDEX + pre-insert check
    nullable: false
    private:  false            # if true: excluded from all DTOs
    default:  null
    values:   []               # required when type: enum
    rules:    # list<Rule> - parameterised validators
      - type: email

```

```

    - type: min_length
      value: 3

queries:                                # declare every DB query needed
  - find_by: [user_id]                  # → GetUsersByUserID(ctx, userID)
  - find_by: [status]
    op: gte                             # enum[eq,neq,gt,gte,lt,lte,between,like,in]
  - exists: [email]                    # → UserExistsByEmail(ctx, email) bool
  - count: [role]                      # → CountUsersByRole(ctx, role) int64
  - sum: [total, user_id]              # → SumTotalByUserID(ctx, userID) decimal
  - paginate: cursor                   # cursor | offset
  order_by:
    - field: created_at
      direction: desc
  default_limit: 20
  - soft_delete: true                  # adds deleted_at, filters everywhere
  - bulk_create: true                  # BatchCreateUsers(ctx, []User)
  - custom: GetUsersWithOrders
  returns: many
  sql: |
    SELECT u.*, COUNT(o.id) as order_count FROM users u
    LEFT JOIN orders o ON o.user_id = u.id
    WHERE u.id = $1 GROUP BY u.id

states:                                # optional state machine
  field: status
  transitions:
    - from: active    to: suspended
    - from: suspended to: active

dtos:                                  # table-level internal model DTO
  model: UserModel                     # (opt) internal DB struct name, default: {Name}

errors: [NotFound, Duplicate] # domain errors for this table

```

3.3 Types Block

Custom value objects used as field types in tables or API DTOs. Always stored as JSONB. No table, no ID, no endpoints.

```

types:
  - name: Money
    fields:
      - name: amount    type: decimal  required: true
      - name: currency  type: str      required: true
    rules:
      - type: min_length
        value: 3
      - type: max_length
        value: 3

  - name: Address
    fields:
      - name: street    type: str      required: true
      - name: city      type: str      required: true
      - name: country   type: str      required: true
      - name: postcode  type: str      nullable: true

```

```
# Use in table fields:
tables:
  - name: orders
    fields:
      - name: total    type: Money    required: true    # JSONB column
      - name: address  type: Address  nullable: true
```

3.4 Transactions Block

Atomic multi-step DB operations. User declares SQL steps. Tool generates full BEGIN/COMMIT/ROLLBACK wrapper. No transaction code written by hand.

```
transactions:
  - name: PlaceOrder                # (req) str - PascalCase function name
    type: local                     # local | saga (saga = Phase 2)
    steps:
      - name: CreateOrder
        sql: |
          INSERT INTO orders (user_id, total, status)
          VALUES ($1, $2, 'pending') RETURNING id
        params:
          - name: user_id  type: int
          - name: total    type: decimal
        error_if_rows: 0
        error: OrderCreationFailed

      - name: DeductInventory
        sql: |
          UPDATE inventory SET quantity = quantity - $1
          WHERE product_id = $2 AND quantity >= $1
        params:
          - name: quantity  type: int
          - name: product_id type: int
        error_if_rows: 0
        error: InsufficientStock
```

3.5 Externals Block

All outbound HTTP dependencies — third-party APIs, inter-service calls, and outbound webhooks. All treated identically: typed client with retry, error mapping, and mock.

```
externals:
  # Inter-service call
  - name: UserServiceClient
    type: http
    base_url: ${USER_SERVICE_URL}
    auth: bearer_token
    timeout: 5s
    retry:
      attempts: 3
      backoff: exponential
```

```

calls:
  - name:      GetUser
    method:    GET
    path:      /users/:id
    response:  UserResponse
  - name:      SuspendUser
    method:    POST
    path:      /users/:id/suspend
    body:      SuspendUserRequest
    response:  void

# Third-party API
- name:      StripeClient
  type:      http
  base_url:  ${STRIPE_URL}
  auth:      bearer_token
  timeout:   10s
  retry:
    attempts: 3
    backoff:   exponential
    on_status: [429, 502, 503]
  headers:
    Stripe-Version: "2023-10-16"
  calls:
    - name:      ChargeCard
      method:    POST
      path:      /v1/charges
      body:      ChargeRequest
      response:  ChargeResponse
      errors:
        - status: 402
          error:  CardDeclined

```

3.6 Messaging Block

Kafka, RabbitMQ, or SQS. Producers are fully generated. Consumers generate the listener loop with DLQ routing — developer implements handler hook body.

```

messaging:
  broker:  kafka
  brokers: [${KAFKA_BROKER}]
  producers:
    - topic: user.created      event: UserCreated      key: user_id
    - topic: order.placed     event: OrderPlaced      key: order_id
  consumers:
    - topic:  payment.completed
      event:  PaymentCompleted
      group:  orders-service
      handler: OnPaymentCompleted
      retry:
        attempts: 5
        backoff:  exponential
        dlq:      payment.failed.dlq

```


3.7 Cache Block

Redis typed cache interfaces. Declared as core infra — the tool generates a typed interface per declared cache with Get/Set/Delete/Invalidate methods. APIs use these via hooks — the developer controls when to read/write through control flags in the shared context.

```
cache:
  provider: redis
  url:      ${REDIS_URL}
  prefix:   orders-svc
  interfaces:
    - name:      UserCache
      key_template: "user:{id}"
      value_type:  UserResponse      # typed - Get returns *UserResponse
      default_ttl: 5m
      methods: [Get, Set, Delete, Invalidate]

    - name:      OrderCache
      key_template: "order:{id}"
      value_type:  OrderResponse
      default_ttl: 2m
      methods: [Get, Set, Invalidate]
```

Tool generates for each cache interface: a typed Go interface, a Redis-backed implementation, and a mock implementation. The interface is injected into every API service that declares a cache touch.

3.8 Resources Block — API Groups

Resources are groups of related API endpoints. Each group has a base path and optional group-level auth. Each API within declares which core infra it touches.

```
resources:
  - group:      UserAPIs          # (req) str - group name
    base_path:  /users            # (req) str - URL prefix
    auth:       jwt               # (opt) group-level default auth

    apis:
      - name:      CreateUser      # (req) str - PascalCase
        method:    POST            # (req) enum[GET, POST, PUT, PATCH, DELETE]
        path:      /              # (req) str - relative to base_path
        auth:      public          # (opt) overrides group auth
        roles:     []              # (opt) list<str> from auth.roles
        owner:     false           # (opt) bool - caller.id == resource FK

      request:    CreateUserRequest # (opt) str - request DTO name
      response:   UserResponse      # (opt) str - response DTO name

      dtos:
        request:
          name: CreateUserRequest
```

```

    fields:
      - name: first_name
        type: str
        required: true
      - name: email
        type: str
        required: true
        rules: [{type: email}]
      - name: password
        type: str
        required: true
        rules: [{type: min_length, value: 8}]
    response:
      name: UserResponse
      fields:
        - id
        - first_name
        - email
        - name: full_name
          type: str
          compute: true      # → ComputeFullName hook stub

    touches:                                # (req) list<Touch> - ordered infra interactions
      - table: users
        op: create
        flag: RunTableUsersCreate
        default: true          # always runs

      - external: CRMClient
        method: CreateContact
        flag: RunCRMClientCreate
        default: false        # conditional - hook sets to true if needed

      - message: UserCreated
        flag: RunMessageUserCreated
        default: true

      - cache: UserCache
        op: invalidate
        flag: RunCacheInvalidate
        default: true

```

3.9 Touch Entry Reference

A touch entry declares one interaction with a core infra system. The order of touch entries defines the execution order in the generated service.

Touch type	Required fields	Generated output
table	table: name, op: create get list update delete	Repo call wired into service. BeforeTable + AfterTable hooks.
external	external: name, method: MethodName	External client call. BeforeExternal + AfterExternal hooks.

message	message: EventName	Typed publish call. BeforeMessage + AfterMessage hooks.
cache	cache: InterfaceName, op: get set delete invalidate	Cache method call. AfterCache hook.
transaction	transaction: TxName	TX orchestrator call. BeforeTx + AfterTx hooks.
storage	storage: upload download, name: UploadName	S3 operation wired in. BeforeStorage + AfterStorage hooks.

Touch flags and defaults

Field	Meaning
flag: RunXxx	Name of the boolean control field in the shared context. Defaults to Run{TouchType}{Name}{Op} if absent.
default: true	Flag starts true — step runs unless a hook sets it false.
default: false	Flag starts false — step does not run unless a hook sets it true. Use for conditional/optional infra calls.

3.10 Shared Context — Generated Structure

The tool generates one shared context struct per API. It is the single carrier of all state through the execution. Developers never create it — the generated service creates it at request entry.

```
// generated/apis/user/createuser_context.go - never edit

type CreateUserContext struct {

    // — Request —————
    Request CreateUserRequest

    // — Control flags —————
    // Set at creation from spec defaults.
    // Hooks set/unset to control execution flow.
    RunTableUsersCreate    bool    // default: true
    RunCRMClientCreate     bool    // default: false (conditional)
    RunMessageUserCreated  bool    // default: true
    RunCacheInvalidate     bool    // default: true

    // — Infra results —————
    // Populated by generated code after each step runs.
    // Available to all subsequent hooks.
    TableUsersResult      *User
    CRMClientResult       *crm.CreateContactResponse

    // — Infra errors —————
```

```

    // Set by generated code. Hook can inspect to decide severity.
    TableUsersError error
    CRMClientError error

    // — Response —————
    // Developer sets in BeforeResponse hook.
    // If nil after hook, tool builds from TableUsersResult.
    Response *UserResponse
}

```

3.11 Hook Interface — Generated Per API

The tool generates one hook interface per API. Every hook receives the full shared context — it can read any previous result and set any future flag. All hooks are optional: nil check before call.

```

// generated/apis/user/createuser_hooks.go - never edit

type CreateUserHooks struct {

    // Entry - inspect request, set initial flags, mutate request (e.g. hash
    password)
    BeforeCreateUser func(ctx context.Context, shared *CreateUserContext) error

    // After table write - read result, set RunCRMClientCreate based on biz rule
    AfterTableUsersCreate func(ctx context.Context, shared *CreateUserContext)
    error

    // After CRM call - inspect CRMClientError, treat as fatal or non-fatal
    AfterCRMClientCreate func(ctx context.Context, shared *CreateUserContext) error

    // Before event publish - modify event payload if needed
    BeforeMessageUserCreated func(ctx context.Context, shared *CreateUserContext)
    error

    // After event published
    AfterMessageUserCreated func(ctx context.Context, shared *CreateUserContext)
    error

    // Build final response - set shared.Response
    // If not set here, tool builds from shared.TableUsersResult
    BeforeResponse func(ctx context.Context, shared *CreateUserContext) error

    // Computed field hooks (from compute: true in DTO)
    ComputeFullName func(ctx context.Context, shared *CreateUserContext) error
}

```

3.12 Generated Service Executor

The generated service reads control flags and calls the appropriate infra. It never makes decisions — it only executes what the flags say. All branching logic lives in developer hooks.

```
// generated/apis/user/createuser_service.go - never edit

func (s *UserAPIService) CreateUser(ctx context.Context, req CreateUserRequest)
(*UserResponse, error) {

    // initialise shared context - flags set to spec defaults
    shared := &CreateUserContext{
        Request:      req,
        RunTableUsersCreate: true,
        RunCRMClientCreate: false,    // conditional - starts false
        RunMessageUserCreated: true,
        RunCacheInvalidate: true,
    }

    // struct-level validation (generated from DTO rules)
    if err := s.validator.Struct(req); err != nil {
        return nil, mapValidationError(err)
    }

    // HOOK: BeforeCreateUser - entry, set flags, mutate request
    if s.hooks.BeforeCreateUser != nil {
        if err := s.hooks.BeforeCreateUser(ctx, shared); err != nil {
            return nil, err
        }
    }

    // STEP: table users create
    if shared.RunTableUsersCreate {
        user, err := s.repo.CreateUser(ctx, fromCreateRequest(shared.Request))
        shared.TableUsersResult = user
        shared.TableUsersError = err
        if err != nil { return nil, fmt.Errorf("create user: %w", err) }

        // HOOK: AfterTableUsersCreate - read result, set RunCRMClientCreate
        if s.hooks.AfterTableUsersCreate != nil {
            if err := s.hooks.AfterTableUsersCreate(ctx, shared); err != nil {
                return nil, err
            }
        }
    }

    // STEP: external CRMClient CreateContact (conditional - default false)
    if shared.RunCRMClientCreate {
        resp, err := s.crmClient.CreateContact(ctx, buildCRMRequest(shared))
        shared.CRMClientResult = resp
        shared.CRMClientError = err
        // note: error does not auto-abort - hook decides severity

        // HOOK: AfterCRMClientCreate - handle CRM result/error
        if s.hooks.AfterCRMClientCreate != nil {
            if err := s.hooks.AfterCRMClientCreate(ctx, shared); err != nil {
                return nil, err
            }
        }
    }

    // STEP: message UserCreated
    if shared.RunMessageUserCreated {
        if s.hooks.BeforeMessageUserCreated != nil {
            if err := s.hooks.BeforeMessageUserCreated(ctx, shared); err != nil {
                return nil, err
            }
        }
    }
}
```

```

    }
    }
    s.producer.PublishUserCreated(ctx, buildUserCreatedEvent(shared))
    if s.hooks.AfterMessageUserCreated != nil {
        s.hooks.AfterMessageUserCreated(ctx, shared)
    }
}

// STEP: cache UserCache invalidate
if shared.RunCacheInvalidate && shared.TableUsersResult != nil {
    s.cache.UserCache.Invalidate(ctx, shared.TableUsersResult.ID)
}

// HOOK: BeforeResponse - build shared.Response
if s.hooks.BeforeResponse != nil {
    if err := s.hooks.BeforeResponse(ctx, shared); err != nil {
        return nil, err
    }
}

// fallback: build response from table result if hook did not set it
if shared.Response == nil && shared.TableUsersResult != nil {
    shared.Response = mapper.ToUserResponse(*shared.TableUsersResult)
    if s.hooks.ComputeFullName != nil {
        s.hooks.ComputeFullName(ctx, shared)
    }
}

return shared.Response, nil
}

```

3.13 Developer Hook Implementation

The developer creates one hooks file per API in the hooks/ directory. This is the only code they write for structural behaviour. All hooks receive the shared context.

```

// hooks/user/createuser.hooks.go - developer owns this entirely

func RegisterCreateUserHooks(svc *UserAPIService) {

    // Entry: validate + mutate request
    svc.CreateUserHooks.BeforeCreateUser = func(ctx context.Context, shared
    *CreateUserContext) error {
        if isDisposableEmail(shared.Request.Email) {
            return ErrDisposableEmail
        }
        // hash password in place on shared request
        hashed, err := bcrypt.GenerateFromPassword([]byte(shared.Request.Password),
        bcrypt.DefaultCost)
        if err != nil { return err }
        shared.Request.Password = string(hashed)
        return nil
    }

    // After DB write: decide whether to sync to CRM

```

```

    svc.CreateUserHooks.AfterTableUsersCreate = func(ctx context.Context, shared
*CreateUserContext) error {
    // only call CRM for business accounts - set the flag
    if shared.TableUsersResult.AccountType == AccountTypeBusiness {
        shared.RunCRMClientCreate = true
    }
    return nil
}

// After CRM: treat CRM failure as non-fatal
svc.CreateUserHooks.AfterCRMClientCreate = func(ctx context.Context, shared
*CreateUserContext) error {
    if shared.CRMClientError != nil {
        log.Warn(ctx, "CRM sync failed, continuing",
            "user_id", shared.TableUsersResult.ID,
            "err", shared.CRMClientError)
        shared.CRMClientError = nil // non-fatal - clear it
    }
    return nil
}

// Build response with computed field
svc.CreateUserHooks.ComputeFullName = func(ctx context.Context, shared
*CreateUserContext) error {
    if shared.Response != nil {
        shared.Response.FullName = shared.TableUsersResult.FirstName + " " +
            shared.TableUsersResult.LastName
    }
    return nil
}
}

```

3.14 Cache Read-Through Pattern

The cache read-through pattern shows how control flags enable short-circuit behaviour. The developer sets `RunTableUsersGet = false` in `BeforeGetUser` after a cache hit. The generated service skips the DB query entirely.

```

# spec - GetUser touches cache first (default false), then table
apis:
  - name:    GetUser
    method:  GET
    path:    /:id
    auth:    jwt
    response: UserResponse
    touches:
      - cache:    UserCache
        op:        get
        flag:      RunCacheRead
        default:   true          # always try cache first

      - table:    users
        op:        get
        flag:      RunTableUsersGet
        default:   true          # hook sets false on cache hit

```

```

- cache:    UserCache
  op:       set
  flag:     RunCacheWrite
  default:  true           # write to cache after DB hit

```

```

// hooks/user/getuser.hooks.go

svc.GetUserHooks.BeforeGetUser = func(ctx context.Context, shared *GetUserContext)
error {
    cached, err := s.cache.UserCache.Get(ctx, shared.Request.ID)
    if err != nil { return err }
    if cached != nil {
        // cache hit - short-circuit
        shared.RunTableUsersGet = false // skip DB
        shared.RunCacheWrite    = false // no need to re-write
        shared.Response         = cached // set response directly
    }
    return nil
}

svc.GetUserHooks.AfterTableUsersGet = func(ctx context.Context, shared
*GetUserContext) error {
    if shared.TableUsersResult != nil {
        resp := mapper.ToUserResponse(*shared.TableUsersResult)
        // RunCacheWrite is true - generated code will call Set after this hook
        shared.Response = &resp
    }
    return nil
}

```

3.15 Extensions and Overrides

Extensions cover full subsystems the DSL cannot express. Overrides change global generated behaviour.

```

extensions:
- name: RealtimeAPIs
  path: extensions/realtime/
  mounts:
    - route: /ws
      middleware: [jwt]
  # Developer implements Extension interface - gets full Dependencies struct
  # (DB, Redis, Kafka, Config, Logger, all generated services)

overrides:
error_format:    custom    # tool calls FormatError(err) hook globally
response_wrapper: custom    # tool calls WrapResponse(data) hook globally
middleware_order: [cors, rate_limit, auth, logging]
uid_type:        uuid      # enum[int, uuid] - default: int
timestamps:      true      # add created_at/updated_at to all tables

```


4. File Ownership and Generated Structure

4.1 File ownership

Directory	Owner	On regenerate
generated/	Tool	Wiped and fully rewritten every time
hooks/	Developer	Never read, never modified by tool
extensions/	Developer	Never read, never modified by tool
custom/	Developer	Never read, never modified by tool

4.2 Generated directory structure

```

generated/
  tables/                                ← one directory per declared table
    {table}/
      model.go                          ← DB struct
      repository.go                     ← all declared query functions
      queries.sql                       ← raw SQL for sqlc
      errors.go                         ← domain error vars
  types/                                ← custom value object structs
    money.go
    address.go
  transactions/                         ← atomic operation orchestrators
    place_order.go
  apis/                                 ← one directory per API
    {api_name}/
      context.go                       ← shared context struct
      hooks.go                         ← hook interface
      service.go                       ← executor – reads flags, calls infra
      handler.go                       ← HTTP layer – parse req, call service
      dto.go                           ← request/response DTOs
      mapper.go                        ← DTO ↔ table model mappers
  externals/                            ← typed HTTP clients
    stripe_client.go
    stripe_client_mock.go
    user_service_client.go
  messaging/                            ← producers + consumer listeners
    producers.go
    consumers.go
  cache/                                ← typed cache interfaces + Redis impl
    user_cache.go
    order_cache.go
  auth/                                 ← JWT middleware + auth endpoints
    middleware.go
    handler.go
  routes.go                            ← all route registration + group prefixes
  wire.go                              ← full DI wiring
  config.go                            ← typed Config + startup validator
  migration/                           ← versioned SQL migrations
    001_create_users.sql
  .stencil.lock                        ← spec snapshot for diff planner

```

4.3 CLI commands

Command	Behaviour
<code>stencil generate</code>	First run — generate all files from spec
<code>stencil update</code>	Diff spec vs lock, regenerate changed files only
<code>stencil diff</code>	Print what would change without writing anything
<code>stencil validate</code>	Parse and validate spec, report all errors, no file output
<code>stencil hooks scaffold <api></code>	Generate empty hooks file for one API
<code>stencil add table <name></code>	Add new table to existing project
<code>stencil add api <group> <name></code>	Add new API to existing group

5. Design Decisions

Decision	What we chose	Rationale
Resources vs tables	Resources = API groups. Tables = infra.	Real APIs touch multiple infra systems. Conflating table with API creates a 1:1 constraint that does not reflect reality.
Shared context	One typed context object per API flows end to end.	Makes all state explicit and inspectable. Eliminates hidden dependencies between hooks. Hooks cannot surprise each other.
Control flags	Boolean fields in shared context control step execution.	Biz logic sets flags, generated code reads flags. Clean separation. No conditional logic in templates.
Conditional default: false	Optional infra touches start as false.	Safe default — nothing unexpected runs. Developer explicitly enables steps they need.
Cache as typed interface	Cache declared in core infra, accessed via hooks.	Cache strategy is business logic. Tool generates the interface. Developer controls when to read/write via flags.
Non-fatal errors	Infra errors stored in context, hook decides severity.	CRM failure should not abort a user creation. Developer sets error=nil in hook to treat as non-fatal.
DTOs on API	Request/response DTOs live on the API definition.	Each API owns its contract. Table model is internal. Mapper is generated per API.
Transactions	Local tx in DSL Phase 1. Sagas Phase 2.	Transaction orchestration is deterministic. Saga orchestrator is complex — defer.
Webhooks	Outbound = externals. Inbound = custom endpoint with auth: hmac.	Webhooks are HTTP with specific auth. No new DSL concept needed.
Inter-service calls	Treated as externals. HTTP only Phase 1.	Same pattern as third-party — typed client, URL from env var.
Multi-tenancy	Not a platform concern. Removed.	Platform-level tenancy breaks cross-tenant use cases. User declares tenant_id as a normal field.

6. Build Roadmap

Phase 1 — Core

- Spec parser + validator — tables, types, externals, messaging, resources, auth, config
- Resolver — defaults, index derivation, FK wiring, shared context struct derivation
- Diff planner + DAG + topological sort
- Table generators — migration, model, repository (sqlc-backed)
- Types generator — custom value object structs
- API generators — shared context, hook interface, service executor, handler, routes, DTOs, mapper
- Auth generator — JWT middleware, login/logout/refresh
- Externals generator — typed HTTP client + mock
- Messaging generator — producers, consumer listeners
- Cache generator — typed interface + Redis impl
- Transaction generator — local tx orchestrator
- Wire generator — DI wiring
- Config generator — typed Config + startup validator
- GoMod generator — go.mod management
- stencil generate + update + diff + validate + hooks scaffold commands

Phase 2 — Integrations

- Saga DSL — distributed transaction orchestrator with compensation hooks
- Storage block — S3/GCS upload endpoints with image transforms
- Scheduled jobs — cron + distributed locking
- SSE block — server-sent event streams
- gRPC transport for externals
- Java Spring Boot parity

Phase 3 — Platform

- Observability — Prometheus, Jaeger, structured logging auto-instrumented
- Extension system with Dependencies struct injection
- Global overrides — error format, response wrapper, middleware order

- WebSocket support — Level 3 extension with PubSub in Dependencies

Phase 4 — Ecosystem

- VS Code extension — spec highlighting, validation, autocomplete
- AI prompt kit — system prompts for Claude Code / Cursor to output Stencil DSL
- Plugin registry — community generators for new frameworks
- Template registry — shareable spec fragments
- Mono-repo workspace support

7. Full DSL Example — Orders Service

A complete stencil.yaml demonstrating all major features.

```

version: 1
project: orders-service
lang: go
framework: gin
db: postgres

# — Config —————
config:
  - name: DATABASE_URL      type: str    required: true
  - name: REDIS_URL         type: str    required: true
  - name: JWT_SECRET        type: str    required: true   min_length: 32
  - name: STRIPE_URL        type: str    required: true
  - name: USER_SERVICE_URL  type: str    required: true
  - name: KAFKA_BROKER      type: str    required: true
  - name: PORT              type: int    default: 8080

# — Core Infra —————
types:
  - name: Money
    fields:
      - name: amount      type: decimal required: true
      - name: currency    type: str      required: true
      rules: [{type: min_length, value: 3}, {type: max_length, value: 3}]

tables:
  - name: users
    fields:
      - name: first_name  type: str    required: true
      - name: last_name   type: str    required: true
      - name: email       type: str    required: true   unique: true
      - name: password_hash type: str    required: true   private: true
      - name: role        type: enum    values: [admin, user] default: user
      - name: status      type: enum    values: [active, suspended] default:
active
      - name: account_type type: enum    values: [personal, business] default:
personal
    queries:
      - find_by: [email]
      - find_by: [status]
      - soft_delete: true
      - paginate: offset
      order_by: [{field: created_at, direction: desc}]
    states:
      field: status
      transitions:
        - from: active    to: suspended
        - from: suspended to: active
      errors: [NotFound, EmailTaken]

  - name: orders
    fields:
      - name: user_id  type: uuid    required: true
      - name: status   type: enum
        values: [pending, confirmed, shipped, delivered, cancelled]
        default: pending
      - name: total    type: Money    required: true

```

```

    - name: items      type: json
  queries:
    - find_by: [user_id, status]
    - soft_delete: true
    - paginate: cursor
      order_by: [{field: created_at, direction: desc}]
  states:
    field: status
    transitions:
      - from: pending    to: confirmed
      - from: confirmed  to: shipped
      - from: shipped     to: delivered
      - from: pending    to: cancelled
    errors: [NotFound, InvalidTransition]

transactions:
  - name: PlaceOrder
    type: local
    steps:
      - name: CreateOrder
        sql: |
          INSERT INTO orders (user_id, total, status)
          VALUES ($1, $2, 'pending') RETURNING id
        params: [{name: user_id, type: uuid}, {name: total, type: Money}]
      - name: DeductInventory
        sql: |
          UPDATE inventory SET quantity = quantity - $1
          WHERE product_id = $2 AND quantity >= $1
        params: [{name: quantity, type: int}, {name: product_id, type: uuid}]
        error_if_rows: 0
        error: InsufficientStock

externals:
  - name: StripeClient
    type: http
    base_url: ${STRIPE_URL}
    auth: bearer_token
    timeout: 10s
    retry: {attempts: 3, backoff: exponential, on_status: [429, 502, 503]}
    calls:
      - name: ChargeCard method: POST path: /v1/charges
        body: ChargeRequest response: ChargeResponse
        errors: [{status: 402, error: CardDeclined}]

  - name: UserServiceClient
    type: http
    base_url: ${USER_SERVICE_URL}
    auth: bearer_token
    timeout: 5s
    retry: {attempts: 3, backoff: exponential}
    calls:
      - name: GetUser method: GET path: /users/:id response: UserResponse

messaging:
  broker: kafka
  brokers: [{KAFKA_BROKER}]
  producers:
    - topic: user.created event: UserCreated key: user_id
    - topic: order.placed event: OrderPlaced key: order_id
  consumers:
    - topic: payment.completed event: PaymentCompleted
      group: orders-service handler: OnPaymentCompleted

```

```

    retry: {attempts: 5, backoff: exponential, dlq: payment.failed.dlq}

cache:
  provider: redis
  url:      ${REDIS_URL}
  prefix:   orders-svc
  interfaces:
    - name: UserCache    key_template: "user:{id}"
      value_type: UserResponse default_ttl: 5m
      methods: [Get, Set, Delete, Invalidate]

auth:
  provider: jwt
  expiry:   24h
  login: {via: [email, password], rate_limit: 5/min}
  refresh_token: {storage: redis}
  password_reset: {via: email, expiry: 1h}
  roles: [admin, user]
  permissions:
    - role: admin  can: [read, write, delete]  on: all
    - role: user   can: [read, write]          on: own

middleware: [logging, recovery, cors, rate_limit: 100rpm]

# — API Layer —————
resources:
  - group:      UserAPIs
    base_path:  /users
    auth:       jwt
    apis:
      - name: CreateUser  method: POST  path: /  auth: public
        dtos:
          request:
            name: CreateUserRequest
            fields:
              - {name: first_name, type: str, required: true}
              - {name: last_name,  type: str, required: true}
              - {name: email,      type: str, required: true, rules: [{type:
email}}}
              - {name: password,   type: str, required: true, rules: [{type:
min_length, value: 8}]}
          response:
            name: UserResponse
            fields: [id, first_name, last_name, email, role]
          touches:
            - {table: users,          op: create, flag: RunTableUsersCreate,
default: true}
            - {external: CRMClient,   method: CreateContact, flag: RunCRMSync,
default: false}
            - {message: UserCreated,  flag: RunMessageUserCreated,
default: true}
            - {cache: UserCache,      op: invalidate, flag:
RunCacheInvalidate,default: true}

      - name: GetUser  method: GET  path: /:id
        dtos:
          response:
            name: UserResponse
            fields: [id, first_name, last_name, email, role, status]
          touches:
            - {cache: UserCache,  op: get, flag: RunCacheRead,      default: true}
            - {table: users,      op: get, flag: RunTableUsersGet,  default: true}

```



```

    - {cache: UserCache, op: set, flag: RunCacheWrite, default: true}

- name: ListUsers method: GET path: / roles: [admin]
  dtos:
    response:
      name: UserSummaryResponse
      fields: [id, first_name, last_name, role, status]
    touches:
      - {table: users, op: list, flag: RunTableUsersList, default: true}

- group: OrderAPIs
  base_path: /orders
  auth: jwt
  apis:
    - name: PlaceOrder method: POST path: /
      dtos:
        request:
          name: PlaceOrderRequest
          fields:
            - {name: items, type: json, required: true}
            - {name: payment_token, type: str, required: true}
        response:
          name: OrderResponse
          fields: [id, status, total]
      touches:
        - {transaction: PlaceOrder, flag: RunTxPlaceOrder, default: true}
        - {external: StripeClient, method: ChargeCard,
          flag: RunStripeCharge, default: true}
        - {message: OrderPlaced, flag: RunMessageOrderPlaced, default: true}

    - name: GetOrder method: GET path: /:id owner: true
      dtos:
        response:
          name: OrderResponse
          fields: [id, user_id, status, total, items, created_at]
      touches:
        - {table: orders, op: get, flag: RunTableOrdersGet, default: true}

- group: PaymentAPIs
  base_path: /payments
  auth: jwt
  apis:
    # Pure external proxy — no DB involved
    - name: ChargeCard method: POST path: /charge
      dtos:
        request:
          name: ChargeCardRequest
          fields:
            - {name: amount, type: Money, required: true}
            - {name: payment_token, type: str, required: true}
        response:
          name: ChargeCardResponse
          fields:
            - {name: charge_id, type: str}
            - {name: status, type: str}
      touches:
        - {external: StripeClient, method: ChargeCard,
          flag: RunStripeCharge, default: true}
        - {message: PaymentCharged, flag: RunMessagePaymentCharged, default:
true}

```